Fashion Item Image Classification: Baseline Neural Network vs Convolutional Neural Network (CNN)

1. Problem Statement

Classifying fashion items from images is an important computer vision task. Manually identifying different types of clothing and accessories can be time-consuming, especially when dealing with large image datasets. Traditional neural networks may have difficulty capturing spatial features in images. Therefore, this project aims to develop and compare a baseline Multilayer Perceptron (MLP) neural network and a Convolutional Neural Network (CNN) to classify fashion images accurately using the Fashion-MNIST dataset.

2. Objective

- To develop an image classification system using deep learning techniques.
- To use the Fashion-MNIST dataset containing 10 categories of fashion items.
- To implement a baseline neural network (MLP) and a CNN.
- To preprocess images by normalizing pixel values and preparing the data for training.
- To train and evaluate both models using accuracy, precision, recall, and F1-score.
- To compare the performance of the two models using graphs and confusion matrices.
- To identify which model performs better for fashion image classification.

3. Scope

This project focuses on classifying grayscale fashion images into 10 categories: T-shirt/top, Trouser, Pullover, Dress, Coat, Sandal, Shirt, Sneaker, Bag, and Ankle boot.

The project includes dataset loading, image preprocessing, training and validation, model development, performance evaluation, visualization, and model saving. The baseline MLP processes flattened image pixels, while the CNN uses convolutional and pooling layers to learn spatial features.

The project is implemented using Python, TensorFlow/Keras, NumPy, Matplotlib, Seaborn, and Scikit-learn. It can be extended in the future using advanced CNN architectures, data augmentation, hyperparameter tuning, and real-world fashion datasets.

4. Implementation Steps

1. Initialization: Import the required libraries, set random seeds, and define the training parameters.
2. Dataset Loading: Load the Fashion-MNIST dataset using TensorFlow/Keras. It contains 70,000 grayscale images of size 28 × 28 pixels, divided into 60,000 training images and 10,000 test images.
3. Data Preprocessing: Normalize pixel values from the range 0–255 to 0–1 and add a channel dimension for CNN processing.
4. Data Splitting: Split the original training data into training and validation sets using an 80:20 ratio.

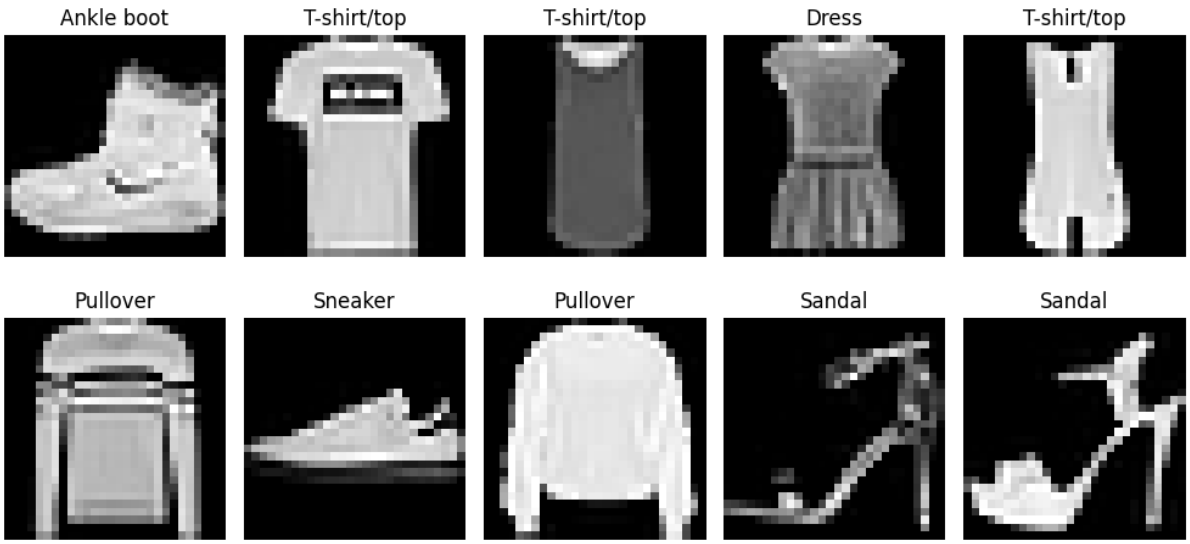
5. Baseline Model Development: Build an MLP using a flattening layer, a dense hidden layer with 128 neurons and ReLU activation, and an output layer with 10 neurons and softmax activation.
6. Baseline Model Training: Train the baseline model for 10 epochs using the Adam optimizer and sparse categorical cross-entropy loss.
7. CNN Model Development: Build a CNN using two convolutional layers, two max-pooling layers, a dense layer, dropout regularization, and a softmax output layer.
8. CNN Model Training: Train the CNN using the same training and validation datasets for a fair comparison.
9. Performance Evaluation: Evaluate both models on the test dataset using accuracy, macro precision, macro recall, and macro F1-score.
10. Visualization: Generate training accuracy and loss graphs, confusion matrices, a model comparison chart, and a sample image prediction.
11. Model Comparison: Compare the evaluation metrics and parameter counts of both models to identify their relative performance.
12. Model Saving: Save the trained models as baseline_model.keras and fashion_cnn_model.keras for future use.

5. Output / Results

The project generates the following outputs:

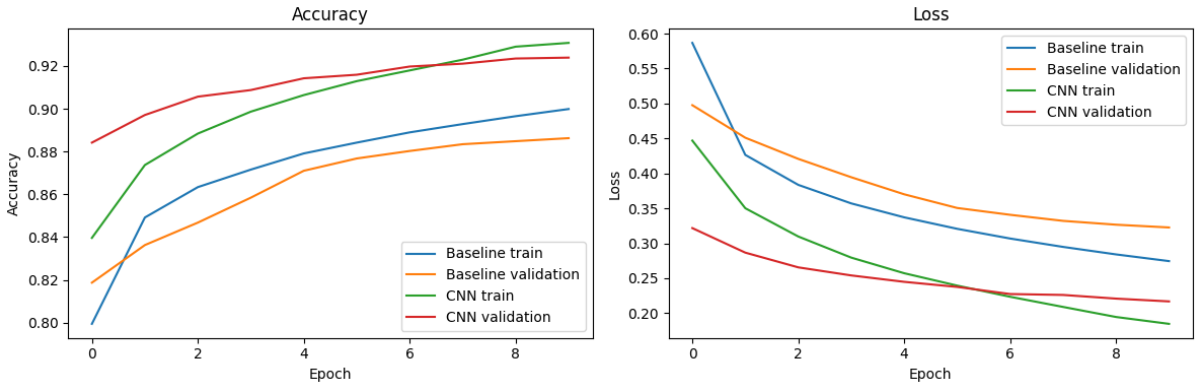
1. Sample Fashion-MNIST Images

Displays sample images from the dataset with their corresponding class labels.



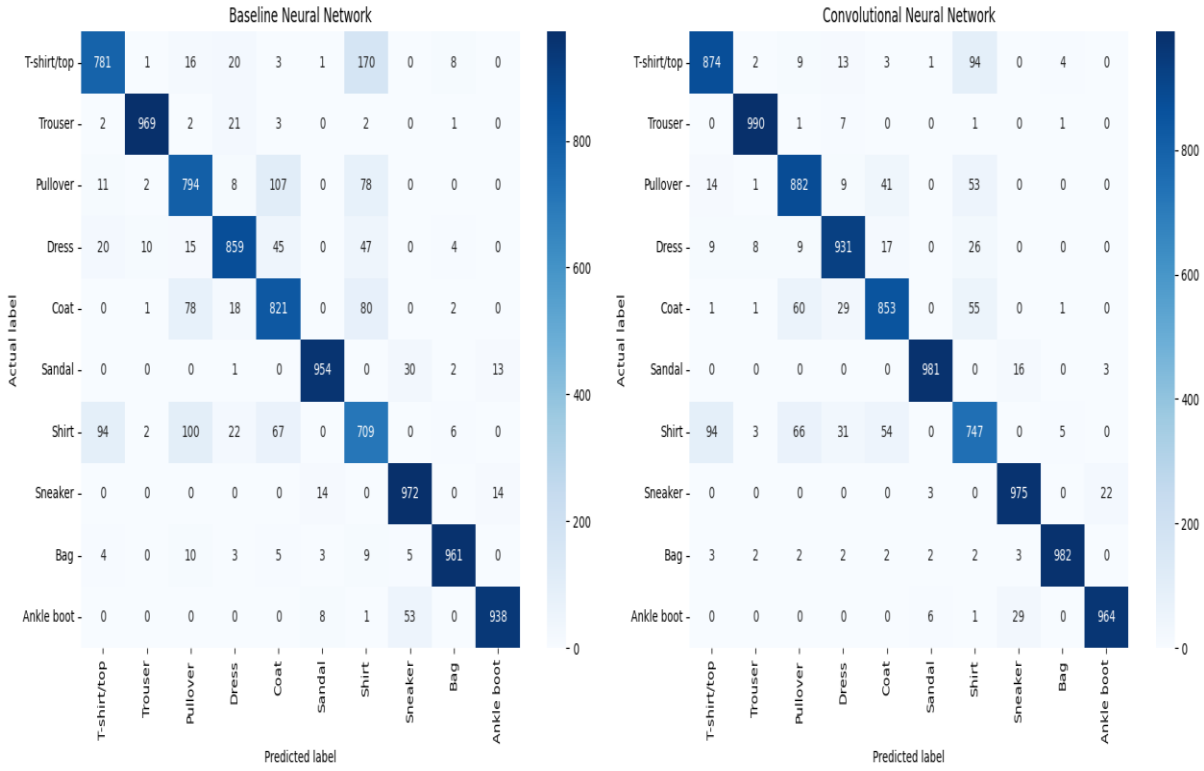
2. Training Performance Graphs

Compares the training and validation accuracy and loss of the baseline MLP and CNN over 10 epochs.



3. Confusion Matrices

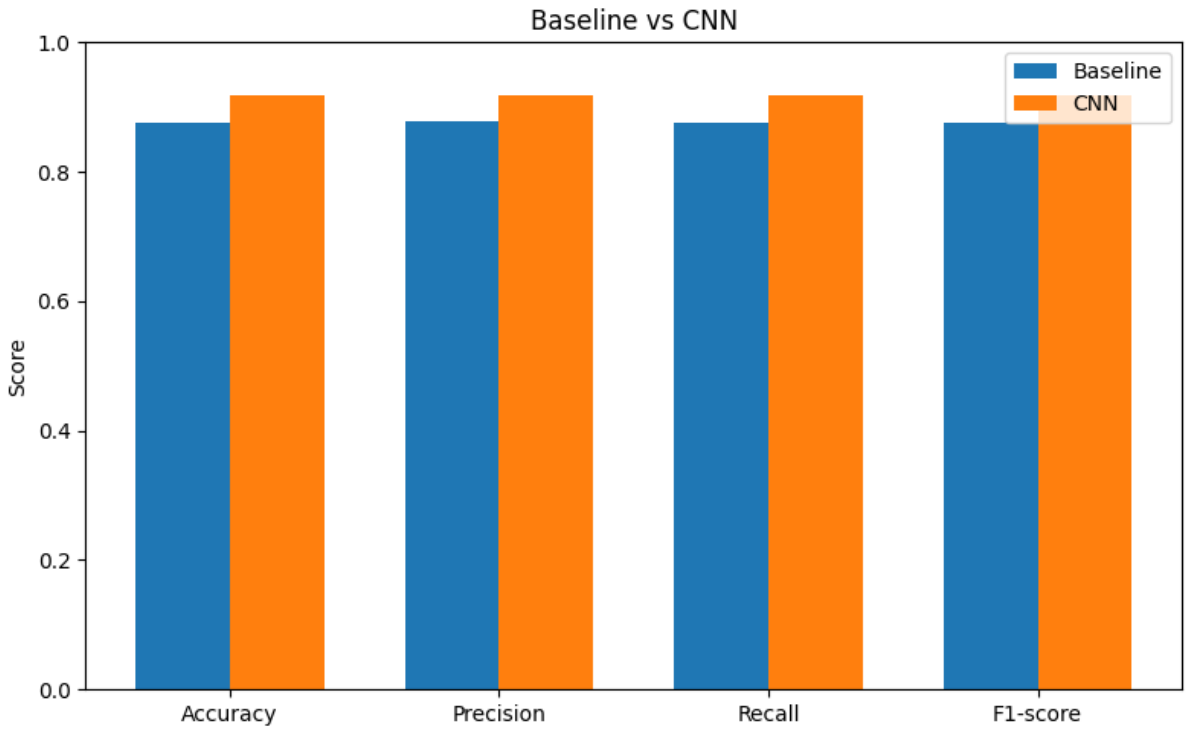
Shows correct and incorrect predictions for each fashion category for both models.



4. Model Comparison Graph

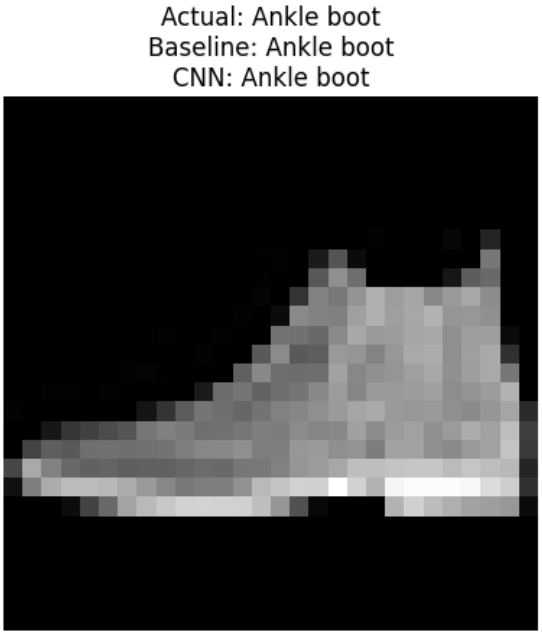
Compares accuracy, precision, recall, and F1-score for the baseline MLP and CNN.

Metric	Baseline	CNN
Accuracy	0.8758	0.9179
Precision	0.8792	0.9177
Recall	0.8758	0.9179
F1-score	0.8769	0.9177



5. Sample Image Prediction

Displays an individual test image with its actual label, predicted labels, and model confidence scores.



Actual class: Ankle boot

<function print(*args, sep=' ', end='\n', file=None, flush=False)>

Evaluation Metrics

Metric	Purpose
Accuracy	Measures the proportion of correctly classified images.
Precision	Measures how often predictions for a class are correct.
Recall	Measures how many actual instances of a class are identified correctly.
F1-score	Balances precision and recall.
Confusion matrix	Identifies correct predictions and misclassifications across classes.

The experiment also saves the two trained models in .keras format. The actual performance values will be obtained after executing the code.

6. Conclusion:

1. The CNN scores higher than the baseline MLP on every metric above, because

   convolution layers look at small local patches and learn edges, textures and shapes, while the MLP flattens the image and loses the 2-D structure.

2. Both models find upper-body items (Shirt, T-shirt, Pullover, Coat) hardest to separate, since they look alike; footwear, Trouser and Bag are easy.

3. Check training_curves.png: a small gap between training and validation accuracy means little overfitting. Dropout in the CNN is intended to help with this. Training for more epochs or adding data augmentation could improve the CNN further.